

8.1.3 答案与解析

一、单项选择题

01. C

拓扑排序是将有向图中所有结点排成一个线性序列，虽然也是在内存中进行的，但它不属于我们这里所提到的内部排序范畴，也不满足前面排序的定义。

02. A

注意，这里的绝对位置是指若在排序前元素 R 在位置 i ，则绝对位置就是 i ，即排序后 R 的位置不发生变化，显然 B 是不对的。C、D 与题目要求无关。

03. B

算法的稳定性与算法优劣无关，A 排除。使用链表也可以进行排序，只是有些排序算法不再适用，因为这时定位元素只能顺序逐链查找，如折半插入排序。

04. A

对于任意序列进行基于比较的排序，求至少的比较次数应考虑最坏情况。对任意 n 个关键字排序的比较次数至少为 $\lceil \log_2(n!) \rceil$ 。将 $n=7$ 代入公式，答案为 13。

上述公式的证明：在基于比较的排序算法中，每次比较两个关键字后，仅出现两种可能的转移。假设整个排序过程至少要做 t 次比较，显然会有 2^t 种情况。由于 n 个记录共有 $n!$ 种不同的排列，因此有 $n!$ 种不同的比较路径，于是有 $2^t \geq n!$ ，即 $t \geq \log_2(n!)$ 。考虑到 t 为整数，所以比较次数为 $\lceil \log_2(n!) \rceil$ 。

8.2.5 答案与解析

一、单项选择题

01. B

直接插入排序在最坏的情况下要做 $n(n-1)/2$ 次关键字的比较, 当 $n=5$ 时, 关键字的比较次数为 10。注意不考虑与哨兵的比较。

02. A

由于序列初始基本有序, 因此使用直接插入排序算法的时间复杂度接近 $O(n)$, 而使用其他算法的时间复杂度均大于 $O(n)$ 。

03. B

由于大部分图书都是有序的，因此采用直接插入排序比较合适。

04. D、A

待排序表为反序时，直接插入排序需要进行 $n(n-1)/2$ 次比较(从前往后依次需要比较 1, 2, ..., $n-1$ 次)；待排序表为正序时，只需进行 $n-1$ 次比较。注意本题不考虑与哨兵的比较。

05. C

冒泡排序和选择排序经过两趟排序后，应该有两个最大(或最小)元素放在其最终位置；插入排序经过两趟排序后，前三个元素应该是局部有序的。只可能是插入排序。

注 意

在排序过程中，每趟都能确定一个元素在其最终位置的有冒泡排序、简单选择排序、堆排序、快速排序，其中前三者能形成全局有序的子序列，后者能确定枢轴元素的最终位置。

06. B

越接近正序的序列，直接插入排序的比较次数就越少。B 和 C 是比较接近正序的，然后分别判断两个序列的比较次数，以 B 为例：第一趟，插入 32，比较 1 次；第二趟，插入 46，比较 1 次；第三趟，插入 40，因为 40 比 46 小但比 32 大，所以比较 2 次；第四趟，插入 80，比较 1 次；第五趟，插入 69，比较 2 次；以此类推，共比较 9 次。同理求出 C 的比较次数为 11 次。所以选 B 项。

07. C

在直接插入排序中，若待排序列中的最后一个元素应插入表中的第一个位置，则前面的有序子序列中的所有元素都不在最终位置上。

08. A

希尔排序是对直接插入排序算法改进后提出来的，本质上仍属于插入排序的范畴。

09. B

希尔排序将序列分成若干组，记录只在组内进行交换。由观察可知，经过一趟后 9 和 -1 交换，7 和 4 交换，可知增量为 4。

10. C

前两个元素已经局部有序，很明显一趟直接插入排序算法有效。再排除其他算法即可。

11. A

增量为 4 意味着所有相距为 4 的记录构成一组，然后在组内进行直接插入排序，经观察，只有 A 项满足要求。

12. B

第一趟：EE 为一组，比较；AS 为一组，比较；ST 为一组，比较；YI 为一组，比较后交换；QO 为一组，比较后交换；UN 为一组，比较后交换，结果为 EASIONESTYQU。第二趟：EIEY 为一组，用直接插入排序需要依次比较 I 和 E、E 和 I、E 和 E、Y 和 I；AOSQ 为一组，依次比较 O 和 A、S 和 O、Q 和 S、Q 和 O；SNTU 为一组，依次比较 N 和 S、T 和 S、U 和 T。第一趟比较次数为 6，第二趟比较次数为 11，总比较次数为 17。

13. B

第一趟增量 $d=5$ ，第一趟排序后，结果为 2, 11, 5, 1, 8, 9, 24, 7, 34, 51, 13, 77, 56。第二趟增量 $d=3$ ，第二趟排序后，结果为 1, 7, 5, 2, 8, 9, 24, 11, 34, 51, 13, 77, 56。

14. C

虽然折半插入排序是对直接插入排序的改进，但它改进的只是比较的次数，而移动次数未发

生变化, 时间复杂度仍为 $O(n^2)$ 。

15. A

因为希尔排序是基于插入排序算法提出的, 所以它不一定在每趟排序过程后将某一元素放置到最终位置上。

16. C

希尔排序是一种复杂的插入排序算法, 它是一种不稳定的排序算法。

17. C

基于插入、交换、选择的三类排序算法中, 通常简单方法是稳定的 (直接插入、折半插入、冒泡), 但有一个例外就是简单选择, 复杂方法都是不稳定的 (希尔排序、快速排序、堆排序)。

18. D

折半插入排序与直接插入排序都将待插入元素插入前面的有序子表, 区别是: 确定当前记录在前面有序子表中的位置时, 直接插入排序采用顺序查找法, 而折半插入排序采用折半查找法。排序的总趟数取决于元素个数 n , 两者都是 $n-1$ 趟。元素的移动次数都取决于初始序列, 两者相同。使用辅助空间的数量也都是 $O(1)$ 。折半插入排序的比较次数与序列初态无关, 时间复杂度为 $O(n \log_2 n)$; 而直接插入排序的比较次数与序列初态有关, 时间复杂度为 $O(n) \sim O(n^2)$ 。

19. B

首先, 第二个元素为 1, 是整个序列中的最小元素, 可知该希尔排序为从小到大排序。然后考虑增量问题, 若增量为 2, 则第 $1+2$ 个元素 4 明显比第 1 个元素 9 要小, 排除 A。若增量为 3, 则第 $i, i+3, i+6$ ($i=1, 2, 3$) 个元素都为有序序列, 符合希尔排序的特点。若增量为 4, 则第 1 个元素 9 比第 $1+4$ 个元素 7 要大, 排除 C。若增量为 5, 则第 1 个元素 9 比第 $1+5$ 个元素 8 要大, 排除 D。

20. A

希尔排序的思想是: 先将待排元素序列分割成若干子序列 (由相隔某个“增量”的元素组成), 分别进行直接插入排序, 然后依次缩减增量再进行排序, 待整个序列中的元素基本有序 (增量足够小) 时, 再对全体元素进行一次直接插入排序。

21. D

如下图所示。

初始序列: 8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6

第一趟: 1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8

第二趟: 1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9

第一趟分组: 8, 1, 6; 3, 4; 9, 7; 11, 5; 2, 10; 间隔为 5, 排序后组内递增。

第二趟分组: 1, 5, 4, 10; 3, 2, 9, 8; 7, 6, 11; 间隔为 3, 排序后组内递增。

因此, 选择选项 D。

8.3.4 答案与解析

一、单项选择题

01. A

冒泡排序最少进行 1 趟冒泡，最多进行 $n-1$ 趟冒泡。初始序列为逆序时，需进行 $n-1$ 趟冒泡，并且元素交换的次数最多。初始序列为正序时，进行 1 趟冒泡（无交换）就可结束算法。

02. C

冒泡排序始终在调整“逆序”，因此交换次数为排列中逆序的个数。对逆序序列进行冒泡排序，每个元素向后调整时都需要进行比较，因此共需要比较 $5+4+3+2+1=15$ 次。

03. D

选择排序在每趟结束后可以确定一个元素的最终位置，不对。插入排序，第 i 趟后前 $i+1$ 个元素应该是有序的，不对。第二趟 $\{20, 15\}$ 和 $\{21, 25\}$ 是反序的，因此不是归并排序。快速排序每趟都将基准元素放在其最终位置，然后以它为准将序列划分为两个子序列。观察题中的排序过程，可知是快速排序。

04. C

以 46 为基准元素，首先从后往前扫描比 46 小的元素，并与之进行交换，而后从前往后扫描比 46 大的元素并将 46 与该元素交换，得到 $(40, 46, 56, 38, 79, 84)$ 。此后，继续重复从后往前扫描与从前往后扫描的操作，直到 46 处于最终位置。

05. D

当待排序数据为基本有序时，每次选取第 n 个元素为基准，会导致划分区间分配不均匀，不利于发挥快速排序算法的优势。相反，当待排序数据分布较为随机时，基准元素能将序列划分为两个长度大致相等的序列，这时才能发挥快速排序的优势。

06. D

这里问的是平均性能，选项 A、B 的平均性能都会达到 $O(n^2)$ ，而希尔排序虽然大大降低了直接插入排序的时间复杂度，但其平均性能不如快速排序。另外，虽然众多排序算法的平均时间复杂度也是 $O(n\log_2 n)$ ，但快速排序算法的常数因子是最小的。

07. A

若为插入排序，则前三个元素应该是有序的，显然不对。而冒泡排序和选择排序经过两趟排序后应该有两个元素处于最终位置（最左/右端），无论是按从小到大还是从大到小排序，数据序列中都没有两个满足这样的条件的元素，因此只可能选 A 项。

【另解】先写出排好序的序列，并和题中的序列做对比。

题中序列：2 1 4 9 8 10 6 20

已排好序序列：1 2 4 6 8 9 10 20

在已排好序的序列中,与题中序列相同的元素有 4、8 和 20,最左和最右两个元素与题中的序列不同,所以不可能是冒泡排序、选择排序或插入排序。

08. C

从后往前冒泡的过程为,第一趟{1, 8, 9, 10, 4, 5, 6, 20, 2},第二趟{1, 2, 8, 9, 10, 4, 5, 6, 20},第三趟{1, 2, 4, 8, 9, 10, 5, 6, 20},第四趟{1, 2, 4, 5, 8, 9, 10, 6, 20},第五趟{1, 2, 4, 5, 6, 8, 9, 10, 20},经过第五趟冒泡后,序列已经全局有序,所以选择选项 C。实际每趟冒泡发生交换后可以判断是否会产生新的逆序对,若不会产生,则本趟冒泡之后序列全局有序,所以最少 5 趟即可。

09. B

第一趟从左往右的排序结果为 4, 7, 3, 5, 6, 8, 9, 1, 2, 10;第二趟从右往左的排序结果为 1, 4, 7, 3, 5, 6, 8, 9, 2, 10;第三趟从左往右的排序结果为 1, 4, 3, 5, 6, 7, 8, 2, 9, 10;第四趟从右往左的排序结果为 1, 2, 4, 3, 5, 6, 7, 8, 9, 10;第五趟从左往右的排序结果为 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,此时序列已有序,但仍需进行一趟无交换的排序才能确定序列已有序,因此共需 6 趟排序。

10. A、D

当每趟的枢轴值都把表等分为长度相近的两个子表时,速度是最快的;当表本身已经有序或逆序时,速度最慢。选项 D 中的序列已按关键字排好序,因此它是最慢的,而选项 A 中第一趟枢轴值 21 将表划分为两个子表{9, 17, 5}和{25, 23, 30},而后对两个子表划分时,枢轴值再次将它们等分,所以该序列是快速排序最优的情况,速度最快。针对其他选项,可以进行类似的分析。

11. B

对各序列分别执行一趟快速排序,可做如下分析(以选项 A 为例):由于枢轴值为 92,因此 35 移动到第一个位置,96 移动到第六个位置,30 移动到第二个位置,再将枢轴值移动到 30 所在的单元,即第五个位置,所以 A 项中序列移动的次数为 4。同样,可以分析出 B 项中序列的移动次数为 8, C 项中序列的移动次数为 4, D 项中序列的移动次数为 2。

12. C

显然,若按从小到大排序,则最终有序的序列是{11, 18, 23, 68, 69, 73, 93};若按从大到小排序,则最终有序的序列是{93, 73, 69, 68, 23, 18, 11}。对比可知选项 I、II 中没有处于最终位置的元素,所以 I、II 项都不可能。III 项中 73 和 93 处于从大到小排序后的最终位置,而且 73 将序列分割成大于 73 和小于 73 的两部分,所以 III 项是有可能的。IV 项中 73 和 93 处于从小到大排列后的最终位置,73 也将序列分割成大于 73 和小于 73 的两部分。

13. B、C

快速排序过程构成一个递归树,递归深度即递归树的高度。枢轴值每次都子表等分时,递归树的高为 $\log_2 n$;枢轴值每次都是子表的最大值或最小值时,递归树退化为单链表,树高为 n 。

14. D

快速排序的最好情况是每次划分将待排序列划分为等长的两部分。因此,第一趟将第 1 个元素与后面的 7 个元素进行比较,将原序列划分为长度为 3 和 4 的两个子表,比较 7 次;第二趟对两个子表进行划分,将长度为 3 的子表划分为长度为 1 的两个子表(不用继续划分),比较 2 次,将长度为 4 的子表划分为长度为 1 和 2 的两个子表,比较 3 次;第三趟将长度为 2 的子表划分为长度为 1 的子表,比较 1 次。至此,排序结束,共进行的比较次数是 $7 + 2 + 3 + 1 = 13$ 。

15. D

快速排序的递归次数与元素的初始排列有关。若每次划分后分区比较平衡,则递归次数少;若划分后分区不平衡,则递归次数多。递归次数与分区处理顺序无关。

16. A

对于绝大部分内部排序而言，只适用于顺序存储结构。快速排序在排序的过程中，既要从后往前查找，也要从前往后查找，因此宜采用顺序存储。

17. C

对 n 个元素进行第一趟快速排序后，会确定一个基准元素，根据这个基准元素在数组中的位置，有两种情况：①基准元素在数组的首端或尾端，接下来对剩下的 $n-1$ 个元素构成的子序列进行第二趟快速排序，再确定一个基准元素。这样，在两趟排序后就至少能确定两个元素的最终位置，其中至少有一个元素是在数组的首端或尾端。②基准元素不在数组的首端或尾端，第二趟快速排序对基准元素划分开的两个子序列分别进行一次划分，两个子序列各确定一个基准元素。这样，两趟排序后就至少能确定三个元素的最终位置。基于上述结论，观察题中的四个选项，A 项的 2, 3, 6, 7, 9 符合第一种或第二种情况；B 项中 2, 9 符合第一种情况；D 项中 5, 9 符合第一种情况；最后看 C 项，只有 9 处于最终位置，因此不可能是快速排序第二趟的结果。

18. D

基于上题中分析得出的结论，观察题中的四个选项，A 项的 28, 72 符合第一种情况；B 项的 2, 72 符合第一种情况；C 项的 2, 28, 32 符合第一种或第二种情况；最后看 D 项，只有 12 和 32 处于最终位置，既不符合第一种情况，又不符合第二种情况。

19. D

第一趟划分后得到的序列中只有一个枢轴，因此可将当前序列和最终排好序的序列进行比较，如下表所示。枢轴会出现在两个序列的相同位置，可以看出枢轴只可能是 77、81，选项只有 81。在当前序列中，77 左边有比它大的元素 80，因此 77 不是枢轴；而 81 左边都是比它小的元素，右边都是比它大的元素，因此 81 是枢轴。

当前序列	68	11	70	23	80	77	48	81	93	88
最终序列	11	23	48	68	70	77	80	81	88	93

8.4.4 答案与解析

一、单项选择题

01. A

02. C

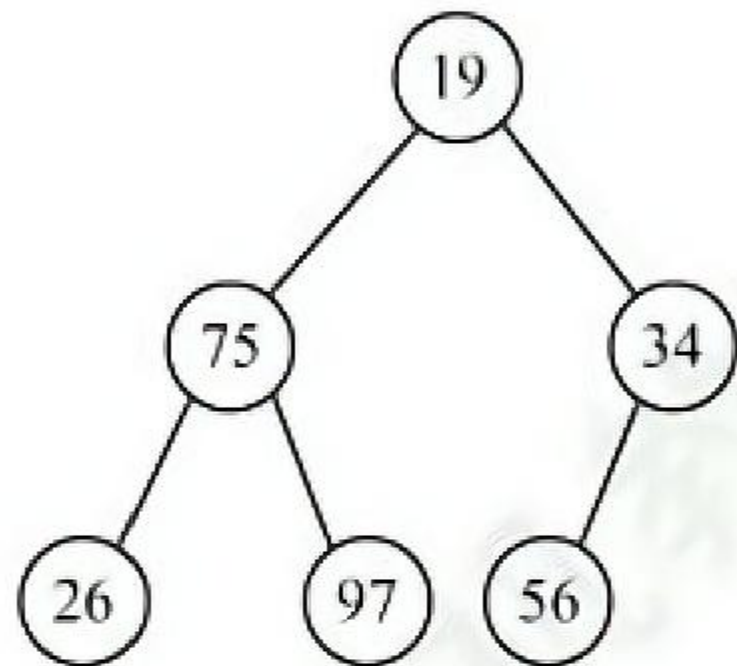
注意，读者应熟练掌握各种排序算法的思想、过程和特点。

03. D

采用堆排序时，读入前 10 个元素，建立含 10 个元素的大根堆，而后依次扫描剩余元素，若大于堆顶，则舍弃，否则用该元素取代堆顶并重新调整堆，当元素全部扫描完毕，堆中保存的即是最小的 10 个元素。冒泡排序需要从后往前执行 10 趟冒泡才能得到 10 个最小的元素。两者的时间复杂度都和数据规模 n 线性相关，但显然堆排序的常系数更小。而快速排序、归并排序的每一趟都不能保证得到当前序列的最小值，也无法达到线性时间复杂度。

04. D

可将每个选项中的序列表示成完全二叉树，再看父结点与子结点的关系是否全部满足堆的定义。例如，项 A 中序列对应的完全二叉树如下图所示。显然，最小元素 19 在根结点，因此可能是小根堆，但 75 与 26 的关系却不满足小根堆的定义，所以项 A 中的序列不是一个堆。其他选项采用类似的过程分析。

**05. B**

这是小根堆，关键字最大的记录一定存储在这个堆所对应的完全二叉树的叶结点中；又因为二叉树中的最后一个非叶结点存储在 $\lfloor n/2 \rfloor$ 中，所以关键字最大记录的存储范围为 $\lfloor n/2 \rfloor + 1 \sim n$ 。

06. C、C

在向有 n 个元素的堆中插入一个新元素时，需要调用一个向上调整的算法，比较次数最多等于树的高度减 1，因为树的高度为 $\lfloor \log_2 n \rfloor + 1$ ，所以堆的向上调整算法的比较次数最多等于 $\lfloor \log_2 n \rfloor$ 。此处需要注意，调整堆和建初始堆的时间复杂度是不一样的，读者可以仔细分析两个算法的具体执行过程。

07. A、D

建堆过程中，向下调整的时间与树高 h 有关，为 $O(h)$ 。每次向下调整时，大部分结点的高度都较小。因此，可以证明在元素个数为 n 的序列上建堆，其时间复杂度为 $O(n)$ 。无论是在最好情况下还是在最坏情况下，堆排序的时间复杂度均为 $O(n \log_2 n)$ 。

08. A

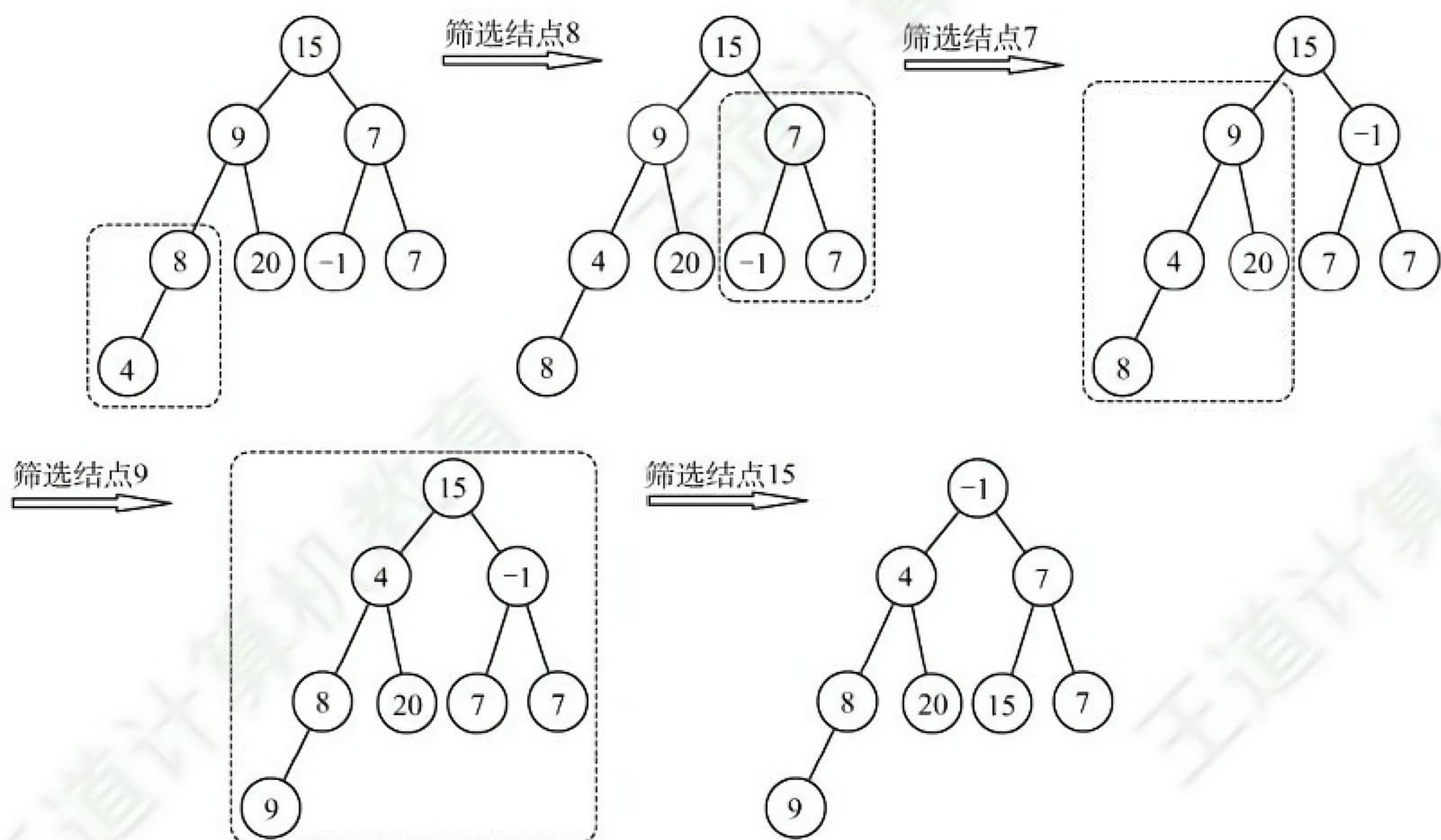
简单选择排序的比较次数始终为 $n(n-1)/2$ ，与序列状态无关。

09. D

堆是顺序存储的完全二叉树，因此其高度小于或等于结点数相同的二叉排序树，A 正确。B 显然正确。根据小根堆的定义，其根结点到任意叶结点的路径构成从小到大的序列，C 正确。堆的各层结点之间没有大小要求，因此层序遍历不能保证得到有序序列，D 错误。

10. C

从 $\lfloor n/2 \rfloor \sim 1$ 依次筛选堆的过程如下图所示，显然选 C 项。



11. D

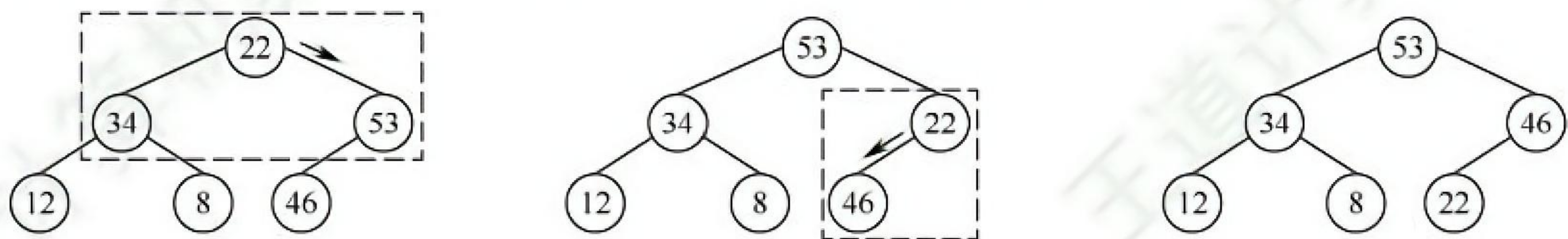
筛选法初始建堆为{8, 17, 23, 52, 25, 72, 68, 71, 60}, 输出 8 后重建的堆为{17, 25, 23, 52, 60, 72, 68, 71}, 输出 17 后重建的堆为{23, 25, 68, 52, 60, 72, 71}。

12. B

初始序列是一棵顺序存储的完全二叉树, 然后根据大根堆的要求, 按照从下到上、从右到左的顺序进行调整。98 和 77 比较, 98 和 77 交换 (交换 1 次); 14 和 35 比较, 35 和 35 比较, 不交换; 98 和 55 比较, 98 和 62 比较, 98 和 62 交换 (交换 1 次); 62 和 77 比较, 77 和 62 交换 (交换 1 次); 98 和 35 比较, 98 和 48 比较, 98 和 48 交换 (交换 1 次); 77 和 55 比较, 77 和 48 比较, 77 和 48 交换 (交换 1 次); 48 和 62 比较, 62 和 48 交换 (交换 1 次), 一共交换 6 次。

13. B

删除堆顶 62 后, 将堆尾 22 放入堆顶, 然后自上而下调。首先 34 与 53 比较 (第一次比较), 较大者 53 与根 22 比较 (第二次比较), 53 被换至堆顶; 22 只有一个孩子, 直接与其左孩子 46 比较 (第 3 次比较), 22 与 46 交换, 至此大根堆调整结束, 具体过程如下图所示。

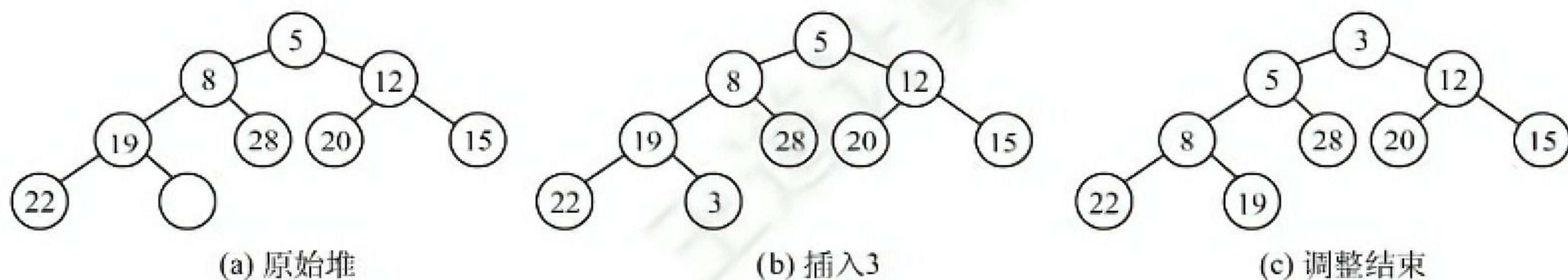


14. D

红黑树和二叉查找树的中序序列是有序序列, 从根结点到任意叶结点的路径不能保证是有序的。哈夫曼树是根据权值按一定规则构造的树, 和关键字次序无关。若是小根堆, 则从根结点到任意叶结点的路径是升序序列; 若是大根堆, 则从根结点到叶结点的路径是降序序列。

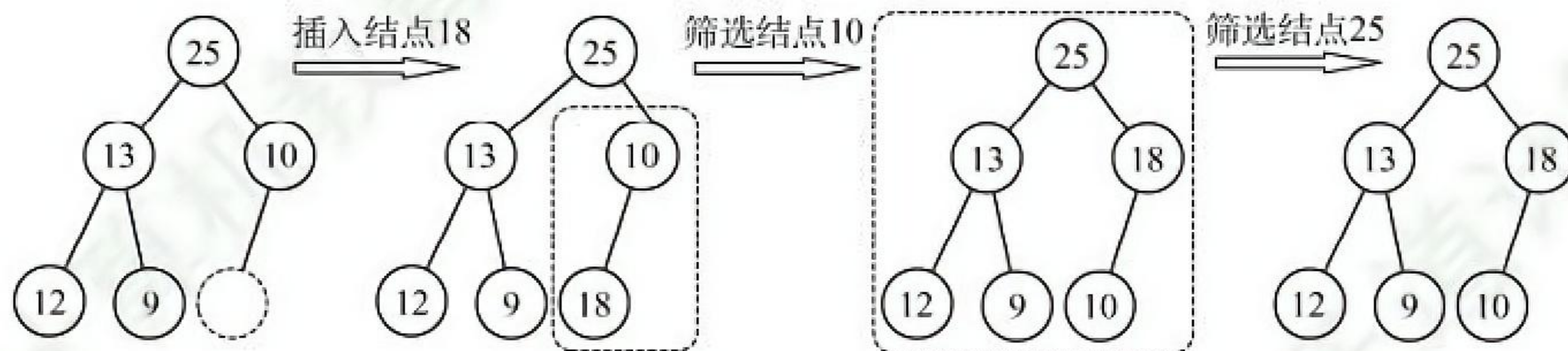
15. A

插入关键字 3 后, 堆的变化过程如下图所示。



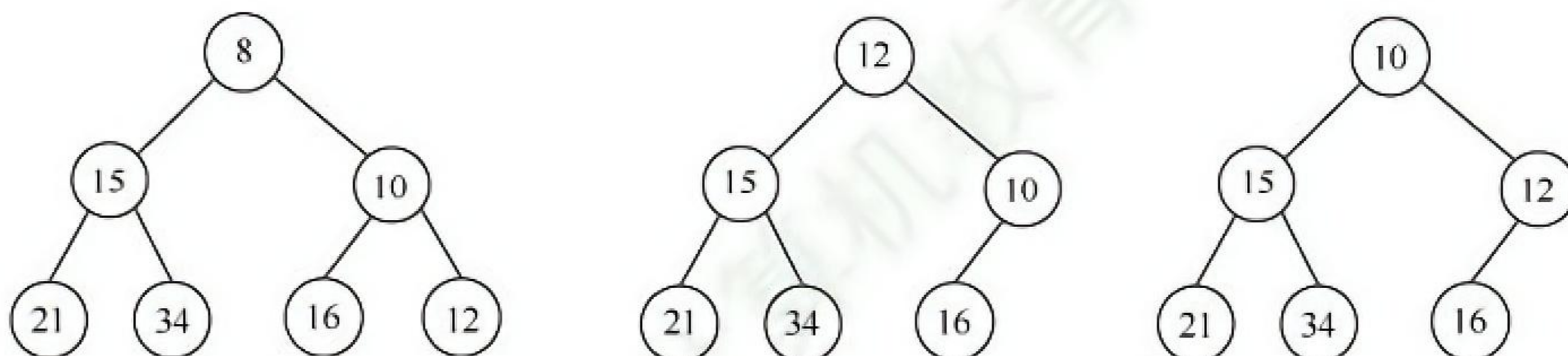
16. B

首先 18 与 10 比较，交换；18 与 25 比较，不交换。共比较 2 次，调整过程如下图所示。



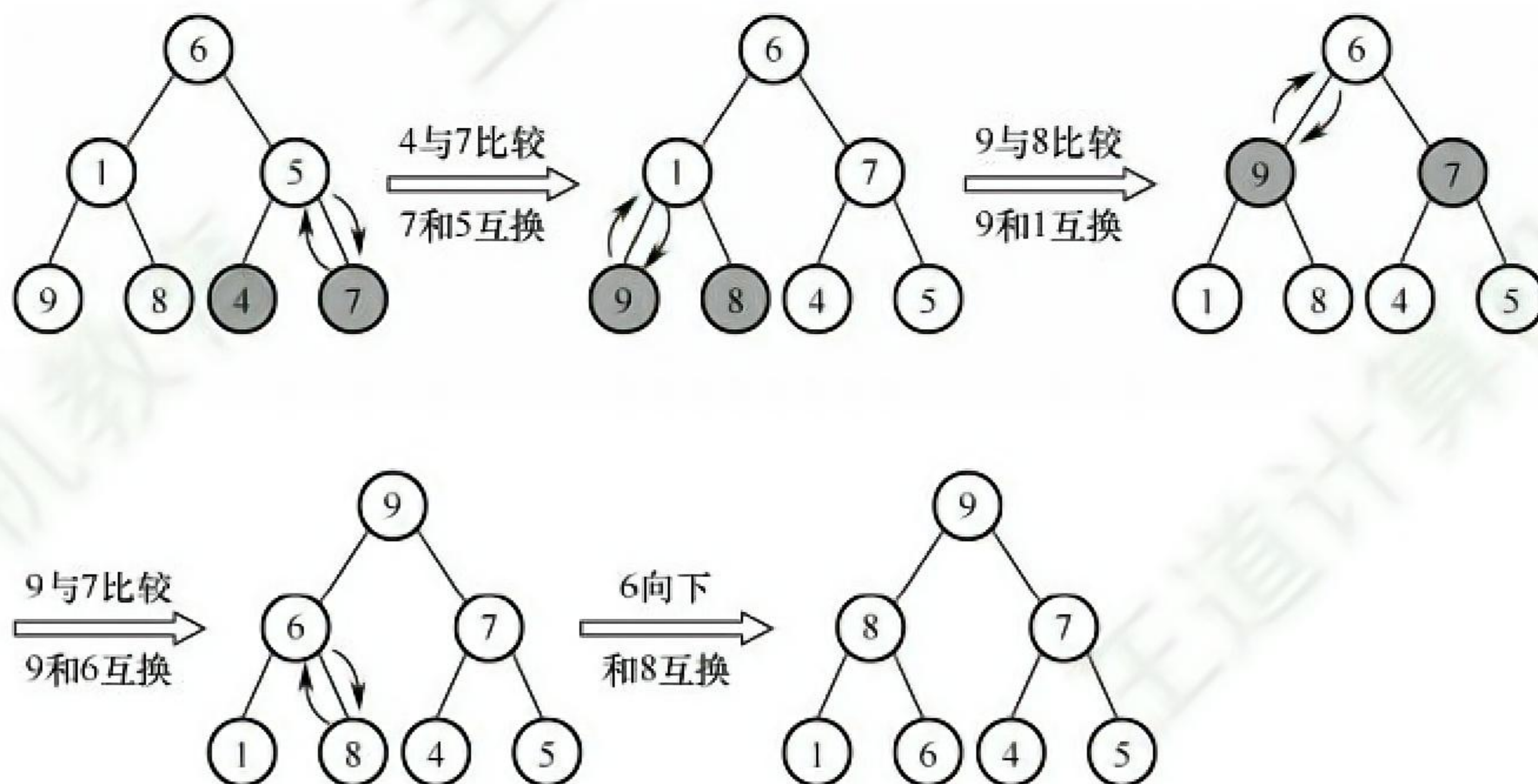
17. C

删除 8 后，将 12 移动到堆顶，第一次是 15 和 10 比较，第二次是 10 和 12 比较并交换，第三次还需比较 12 和 16，所以比较次数为 3。



18. A

要熟练掌握建堆和调整堆的方法，从序列末尾开始向前遍历，变换过程如下图所示。

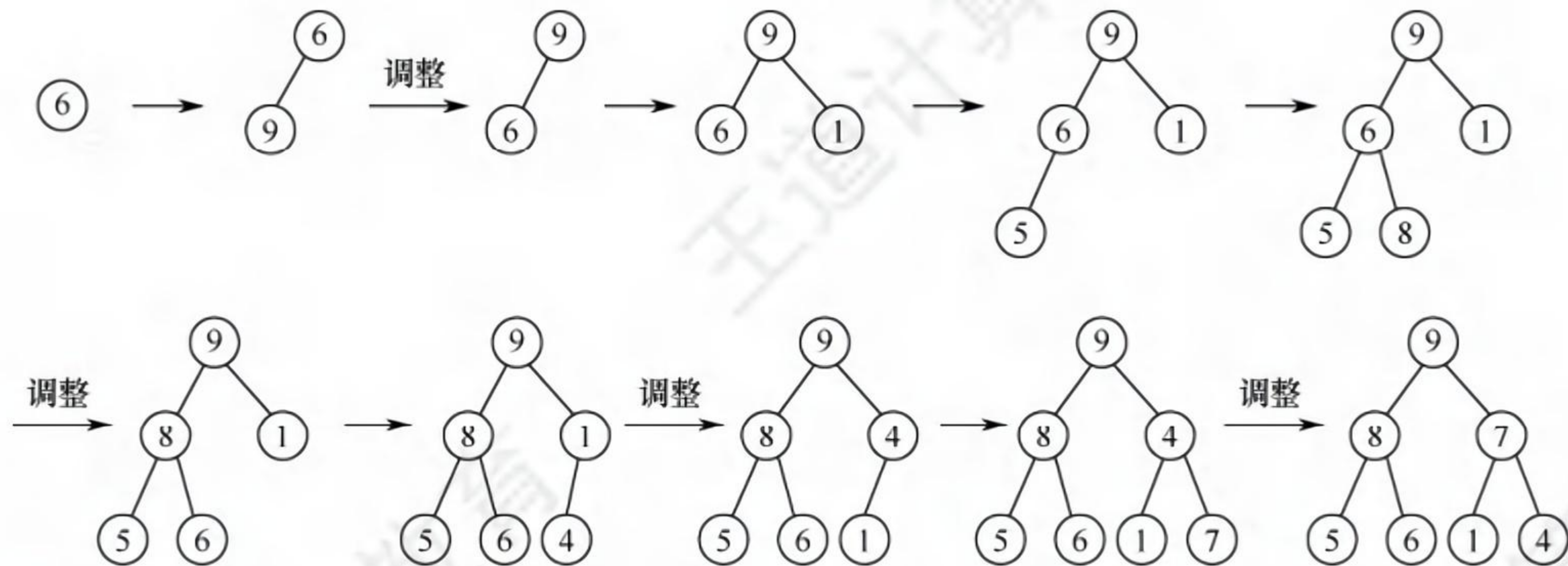


19. C

简单概念题。堆是一棵完全树，采用一维数组存储，I 正确，II 正确。大根堆只要求根结点值大于左右孩子值，并不要求左右孩子值有序，III 错误。堆的定义是递归的，所以其左右子树也是大根堆，所以堆的次大值一定是其左孩子或右孩子，IV 正确。

20. B

要熟练掌握调整堆的方法，建堆的过程如下图所示，所以答案选择选项 B。



注意，给定序列依次插入空堆的结果与给定序列直接调整成堆的结果是不同的，最终得到的堆的形式也不同。若对序列 6, 9, 1, 5, 8, 4, 7 直接调整成堆，则会误选 C 项。

8.5.5 答案与解析

一、单项选择题

01. C

我们知道插入排序不能保证在一趟排序结束后一定有元素放在最终位置上。事实上，归并排序也不能保证。例如，序列{6, 5, 7, 8, 2, 1, 4, 3}进行一趟二路归并排序（从小到大）后为{5, 6, 7, 8, 1, 2, 3, 4}，显然它们都未被放在最终位置上。

02. C

基数排序是基于关键字各位的大小进行排序的，而不是基于关键字的比较进行的。

03. D、C

归并排序在平均情况和最坏情况下的空间复杂度都是 $O(n)$ ，快速排序只在最坏情况下才是 $O(n)$ ，平均情况是 $O(\log_2 n)$ 。因此，归并排序是本章所有排序算法中占用辅助空间最多的。

04. A

前面已讲过选择排序的比较次数与序列初始状态无关，归并排序的比较次数的数量级也与序列的初始状态无关。读者应能从算法的原理方面来考虑为什么和初始状态无关。

05. B

N 个元素进行 k 路归并排序的趟数 m 满足 $k^m = N$ ，即 $m = \lceil \log_k N \rceil$ ，本题中为 $\lceil \log_2 n \rceil$ 。

06. B

N 个元素进行 k 路归并排序的趟数 m 满足 $k^m = N$ ，这里要求的是 k ，代入可得 $k = 3$ 。

07. A、B

注意，当一个表中的最小元素比另一个表中的最大元素还大时，比较的次数是最少的，仅比较 N 次；而当两个表中的元素依次间隔地比较时，即 $a_1 < b_1 < a_2 < b_2 < \dots < a_n < b_n$ 时，比较的次数是最多的，为 $2N - 1$ 次。

建议读者对此举一反三：若将本题中的两个有序表的长度分别设为 M 和 N ，则最多（或最少）的比较次数是多少？时间复杂度又是多少？

08. C

第一趟归并后{1, 2}, {4, 6}, {3, 5}, {7, 8}，共比较 4 次；第二趟归并后{1, 2, 4, 6}, {3, 5, 7, 8}，共比较 4 次；第三趟归并后{1, 2, 3, 4, 5, 6, 7, 8}，共比较 6 次。三趟归并共需进行 14 次比较。

09. B

由于这里采用二路归并排序算法,而且是第二趟排序,因此每4个元素放在一起归并,可将序列划分为{25, 50, 15, 35}, {80, 85, 20, 40}和{36, 70}, 分别对它们进行排序后有{15, 25, 35, 50}, {20, 40, 80, 85}和{36, 70}。

10. D

按照所有中国人的生日排序,一方面 N 是非常大的,另一方面关键字所含的排序码数为2,且一个排序码的基数为12,另一个排序码的基数为31,都是较小的常数值,因此采用基数排序可以在 $O(N)$ 内完成排序过程。

11. D

本题思路来自基数排序的LSD, 首先应确定 k_1 、 k_2 的排序顺序, 若先排 k_1 再排 k_2 , 则排序结果不符合题意, 排除A和C。再考虑排序的稳定性, 当 k_2 排好序后, 再对 k_1 排序, 若对 k_1 排序采用的方法是不稳定的, 则对于 k_1 相同而 k_2 不同的元素可能会改变相对次序, 从而不一定能满足题设要求。直接插入排序是稳定的, 而简单选择排序是不稳定的, 只能选D。

12. C

基数排序有MSD和LSD两种, 基数排序是稳定的。对于A, 不符合LSD和MSD; 对于B, 符合MSD, 但关键字42、46的相对位置发生了变化; 对于D, 不符合LSD和MSD。

13. D

基数排序中建立的队列个数等于进制数。

14. C

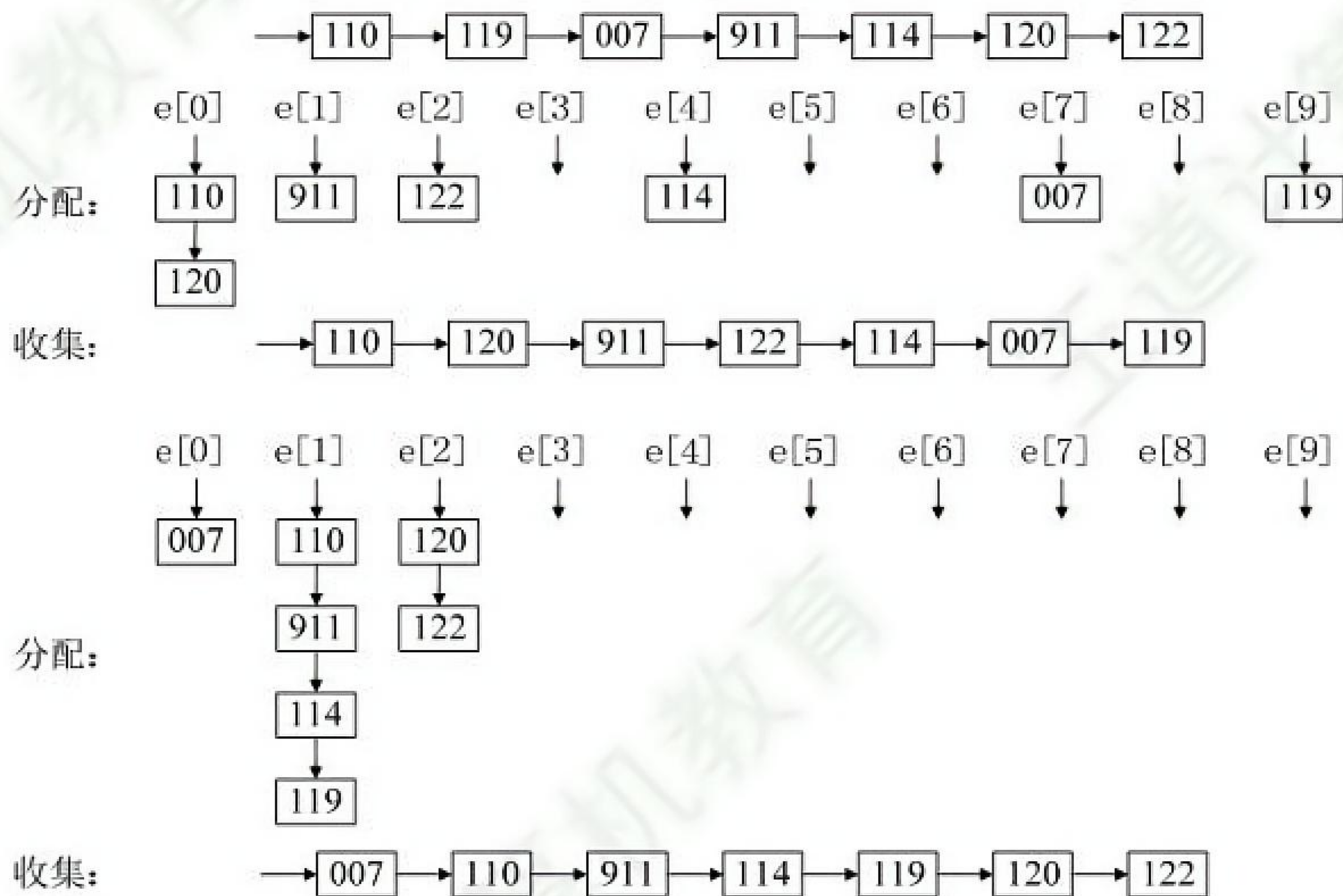
快速排序的平均空间复杂度是 $O(\log n)$, 归并排序的空间复杂度是 $O(n)$, 其他都是 $O(1)$ 。

15. D

考虑两个升序链表合并, 两两比较表中元素, 每比较一次, 确定一个元素的链接位置(取较小元素, 头插法)。当一个链表比较结束后, 将另一个链表的剩余元素插入即可。最坏的情况是两个链表中的元素依次进行比较, 因为 $2\max(m, n) \geq m + n$, 所以时间复杂度为 $O(\max(m, n))$ 。此外, 每次比较把两个链表中的较小结点插入新链表的表头, 直到一个链表为空, 因为原链表是升序排列的, 要求合并后为降序排列的, 因此还要把另一个链表剩下的结点一一插入新链表的表头, 不论是最好情况还是最坏情况, 都需要遍历两个链表中的所有结点。

16. C

基数排序的第一趟排序是按照个位数字的大小来进行的, 第二趟排序是按照十位数字的大小来进行的, 排序的过程如下图所示。



17. C

基数排序的元素移动次数与序列初态无关，而其他三种排序算法都与序列初态有关。

18. C

基数排序是一种稳定的排序算法。由于采用最低位优先（LSD）的基数排序，即第一趟对个位进行分配和收集操作，因此第一趟分配和收集后的结果是{151, 301, 372, 892, 93, 43, 485, 946, 146, 236, 327, 9}，元素 372 之前、之后紧邻的元素分别是 301 和 892。

19. A。

送分题。本书对归并的定义原话是“归并的含义是将两个或两个以上的有序表合并成一个新的有序表”，而二路归并是将两个有序表合并为一个新的有序表。

8.6.4 答案与解析

一、单项选择题

01. A

采用排除法。由于题目要求是稳定排序，因此排除 B 项和 D 项，又由于基数排序不能对 float 和 double 类型的实数进行排序，因此排除 C 项。

02. C

堆排序和快速排序不是稳定排序算法，而直接插入排序算法的时间复杂度为 $O(n^2)$ 。

03. C

各种排序算法的时间和空间复杂度、稳定性等见表 8.1。

04. ① I、IV、VI ② II、VI、VII ③ I、IV

读者应能从算法的原理上理解算法的稳定性情况。堆排序和归并排序在最坏情况下的时间复杂度与最好情况下的时间复杂度是同一数量级的，都是 $O(n\log_2 n)$ 。

05. A

由于堆排序的空间复杂度为 $O(1)$ ，因此快速排序的空间复杂度在最坏情况下为 $O(n)$ ，平均空间复杂度为 $O(\log_2 n)$ ，归并排序的空间复杂度为 $O(n)$ 。

06. B

冒泡排序的趟数为 $1 \sim n-1$ ，和序列初态有关。直接插入排序每趟都插入一个元素，排序趟数固定为 $n-1$ 。简单选择排序每趟都选出一个最小（或最大）的元素，排序趟数固定为 $n-1$ 。基数排序每趟都要进行分配和收集，排序趟数固定为 d 。

07. C

不论序列的原始状态如何，简单选择排序和直接插入排序的趟数都是 $n-1$ 。冒泡排序的趟数为 $1 \sim n-1$ ，快速排序的趟数为 $\log_2 n \sim n-1$ ，具体取决于序列的原始状态（快速排序还取决于划分方法）。

08. A

选择排序和序列初态无关，直接排除。初始序列基本有序时，插入排序比较次数较少。本题中，插入排序仅需比较 $n-1+4$ 次，而希尔排序和冒泡排序的比较次数均远大于此。

09. C

堆是用于排序的，在查找时它是无序的，所以效率没有其他查找结构的高。

10. C

插入排序和选择排序的趟数始终为 $n-1$ ，与序列的原始状态无关。对于冒泡排序，某趟比较后没有发生元素交换，则说明已排好序。基数排序的趟数由元素的位数决定，与原始状态无关。

11. A

简单选择排序的总比较次数显然是确定的。折半插入排序每趟的比较次数都为 $O(\log_2 m)$ (m 为当前已排好序的子序列的长度)，因此总比较次数也是确定的。基数排序不是基于比较的排序算法。其他几种排序算法的比较次数显然和序列的初始状态有关。

12. A

基数排序每趟需要利用前一趟已排好的序列，无法并行执行。快速排序每趟划分的子序列互不影响，可以并行执行。冒泡排序每趟对未排序的所有元素进行一趟处理，无法并行执行。堆排序可以并行执行，因为根结点的左右子树构成的子堆在执行过程中是互不影响的。

13. B

每趟冒泡和选择排序后，总会有一个元素被放置在最终位置上。显然，这里 {11, 12} 和 {4, 5} 所处的位置并不是最终位置，因此不可能是冒泡和选择排序。二路归并算法经过第二趟后应该是每 4 个元素有序的，但 {11, 12, 13, 7} 并非有序，因此也不可能是二路归并排序。

14. A

题中给出的排序过程，每一趟都是从前往后依次比较使最大值沉底，符合冒泡排序的特点。分别用其他 3 种排序算法尝试，归并排序第一趟后的结果为 (2, 12, 16, 88, 5, 10)，基数排序第一趟后的结果为 (10, 2, 12, 5, 16, 88)，希尔排序显然不符合。

15. A

对于 I，简单选择排序每次选择未排序序列中的最小元素放入其最终位置。对于 II，希尔排序每次对划分的子表进行排序，得到局部有序的结果，所以不能保证每趟结束都能确定一个元素的最终位置。对于 III，快速排序每趟结束后都将枢轴元素放到最终位置。对于 IV，堆排序属于选择排序，每次都大根堆的根结点与表尾结点交换，确定其最终位置。对于 V，二路归并排序每趟对子表进行两两归并，从而得到若干局部有序的结果，但无法确定最终位置。

16. B

归并排序的代码比插入排序的代码更为复杂，前者的空间复杂度是 $O(n)$ ，后者是 $O(1)$ 。但是前者的时间复杂度是 $O(n \log n)$ ，后者是 $O(n^2)$ 。

17. D

在顺序存储的条件下，插入排序、选择排序、冒泡排序的时间复杂度都是 $O(n^2)$ ，更换为链式存储后的时间复杂度还是 $O(n^2)$ 。希尔排序和堆排序都利用了顺序存储的随机访问特性，而链式存储不支持这种性质，所以时间复杂度会增加。

18. D

当数据规模较小时可选择复杂度为 $O(n^2)$ 的简单排序算法，当数据规模较大时应选择复杂度为 $O(n \log_2 n)$ 的排序算法，当数据规模大到内存无法放下时需选择外部排序算法，I 正确。数据的存储方式主要分为顺序存储和链式存储，有些排序算法（如堆排序）只能用于顺序存储方式，II 正确。若对数据稳定性有要求，则不能选择不稳定的排序算法，III 显然正确。当数据初始基本有序时，直接插入排序的效率最高，冒泡排序和直接插入排序的时间复杂度都是 $O(n)$ ，而归并排序的时间复杂度依旧是 $O(n \log_2 n)$ ，IV 正确。

19. A

考虑比较极端的情况，对于有序数组，直接插入排序的比较次数为 $n-1$ ，简单选择排序的比

较次数始终为 $1+2+\cdots+n-1=n(n-1)/2$ ，I 正确。两种排序算法的辅助空间都是 $O(1)$ ，无差别，II 错误。初始有序时，移动次数均为 0；对于通常情况，直接插入排序每趟插入都需要依次向后挪位，而简单选择排序只需与找到的最小元素交换位置，后者的移动次数少很多，III 错误。

20. D

直接插入排序和快速排序的特点如下表所示。

	适合初始序列情况	适合元素数量	空间复杂度	稳定性
直接插入排序	大部分元素有序	较少	$O(1)$	稳定
快速排序	基本无序	较多	$O(\log_2 n)$	不稳定

可见，选项 I、II、III、IV 都是采用直接插入排序而不采用快速排序的可能原因。

21. C

稳定的内部排序算法：插入排序、冒泡排序、归并排序和基数排序。不稳定的内部排序算法：简单选择排序、快速排序、希尔排序和堆排序。

8.7.7 答案与解析

一、单项选择题

01. B

外部排序和内部排序最主要的区别是是否涉及内存、外存的数据交换。

02. C

置换选择排序得到的是长度不一定相等的归并段, A 错误。外部排序的主要时间消耗在内外存之间的数据交换上, B 错误。败者树是一棵完全二叉树, C 正确。外部排序包括两个阶段: 生成初始归并段和对初始归并段进行归并, 这两个阶段都涉及对文件的读/写操作, D 错误。

03. A

多路平衡归并的目的是减少归并趟数, 因为当 m 个初始归并段采用 k 路平衡归并时, 所需趟数 $s = \lceil \log_k m \rceil$, 若不采用多路平衡归并, 则其归并趟数大于 s 。

04. B

初始归并段的个数 $r = 375000/600 = 625$, 因此, 归并趟数 $S = \lceil \log_m r \rceil = \lceil \log_5 625 \rceil = 4$ 。第一趟把 625 个归并段归并成 $625/5 = 125$ 个; 第二趟把 125 个归并段归并成 $125/5 = 25$ 个; 第三趟把 25 个归并段归并成 $25/5 = 5$ 个; 第四趟把 5 个归并段归并成 $5/5 = 1$ 个。

05. D

在外部排序过程中输入/输出缓冲区就是排序的内存工作区, 例如做 m 路平衡归并需要 m 个输入缓冲区和 1 个输出缓冲区, 用以存放参加归并的和归并完成的记录。在产生初始归并段时也可用作内部排序的工作区。它没有传送用户界面的消息的任务。

06. D、A

相比普通的 m 路归并: 需增加一个输出缓冲区, 当一个输出缓冲区满时, 输出一个缓冲区的同时归并程序可向另一个输出缓冲区填充数据, 这就实现了内部归并和输出的并行。需增加 m 个输入缓冲区, 当 m 个输入缓冲区正在运行时, 外部可向新增的 m 个缓冲区写入数据, 这就实现了输入和内部归并的并行。综上, 需设置 2 个输出缓冲区, $2m$ 个输入缓冲区。

07. C

归并趟数 $= \lceil \log_k n \rceil$, 其中 k 表示归并的路数, n 表示元素个数, 当 $k = 4$ 、 $n = 64$ 时, 归并趟数恰好等于 3, 因此选取的归并路数至少是 4。

08. A

置换-选择排序是外部排序中生成初始归并段的方法, 用此方法得到的初始归并段的长度是不等长的, 其长度平均是传统等长初始归并段的 2 倍, 从而使得初始归并段数减少到原来的近二分之一。但是, 置换-选择排序不是一种完整的生成有序文件的外部排序算法。

09. D

设内存工作区 $w = 1$, 则文件 $\{1, 2, 3, 4, 5\}$ 产生 1 个有序段, 而文件 $\{5, 4, 3, 2, 1\}$ 产生 5 个有序段, 因此 m 与待排文件、内存工作区大小 w 和 n 都有关, 但不是 A、B、C 项描述的直接关系。

10. D

在败者树中选取最小关键字的时间复杂度取决于败者树的高度, 所需时间为 $O(\log k)$ 。

11. D

I 正确, 是败者树的性质。在败者树的维护过程中, 会让胜利者一直调整到根结点, II 正确。以小根堆为例, 每次调整时, 先比较下一层的两个元素 (1 次), 找出较小值, 然后比较当前元素和下一层的较小元素 (1 次), 以决定是否向下交换位置, III 正确。堆在维护时, 可能会在中间某层停止 (若此处无须调整), 而不一定要走到叶结点, IV 错误。

12. B

最佳归并树在外部排序中的作用是设计 m 路归并排序的优化方案, 仿照构造哈夫曼树的方法, 以初始归并段的长度为权值, 构造具有最小带权路径长度的 m 叉哈夫曼树, 可以有效地减少归并过程中的读/写记录数, 加快外部排序的速度。

13. C

k 阶最佳归并树中只有度为 0 和 k 的结点。设结点总数为 n , 度为 0 的结点数为 n_0 , 度为 k

的结点数为 n_k , 则 $n_0 = m$ 、 $n - 1 = k \times n_k$, $n = n_0 + n_k$, 因此 $k \times n_k = m + n_k - 1$, 求得 $n_k = (m - 1)/(k - 1)$ 。

14. B

题中的三叉树为使 WPL 最小, 必须构造三叉哈夫曼树, 应满足 $(n_0 - 1) \% (3 - 1) = 0$ 的条件, 因此需添加 1 个权值为 0 的虚叶结点, 说明 7 个叶结点刚好可构成一棵严格的三叉树。按照哈夫曼树的原则, 权为 0 的叶结点应离树根最远, 构造三叉哈夫曼树的过程如下:

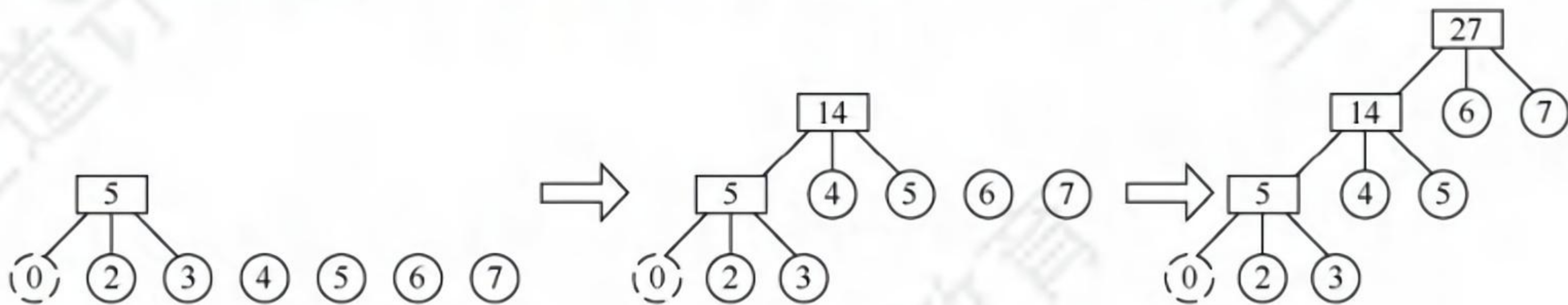
- ① 合并权值最小的三个结点 0, 2, 3, 得到新结点的权值 = 5, 剩下 5, 4, 5, 6, 7。
- ② 合并权值最小的三个结点 4, 5, 5, 得到新结点的权值 = 14, 剩下 14, 6, 7。
- ③ 合并权值最小的三个结点 6, 7, 14, 得到新结点的权值 = 27, 仅有 27, 建树完成。

$$\text{WPL} = \sum_{i=1}^n \text{权值 } W_{\text{叶结点 } i} \times \text{深度 } D_{\text{叶结点 } i} = (2 + 3) \times 3 + (4 + 5) \times 2 + (6 + 7) \times 1 = 46$$

或

$$\text{WPL} = \sum_{i=1}^m \text{权值 } W_{\text{分支结点 } i} = 27 + 14 + 5 = 46$$

每个分支结点的权值都累加了其下面所有分支结点的权值, 因此采用第二种方法更方便。



15. D

外部排序指的是大文件的排序, 即待排序的记录存储在外存中, 待排序的文件无法一次性装入内存, 需要在内存和外存之间进行多次数据交换, 以达到排序整个文件的目的。外部排序通常采用归并排序算法。A、B、C 项都是内部排序的方法。

16. B

在 12 路归并树中只存在度为 0 和度为 12 的结点, 设度为 0 的结点数、度为 12 的结点数和要补充的结点数分别为 n_0 、 n_{12} 和 $n_{\text{补}}$, 则有 $n_0 = 120 + n_{\text{补}}$, $n_0 = (12 - 1)n_{12} + 1$, 可得 $n_{12} = (120 - 1 + n_{\text{补}})/(12 - 1)$ 。因为结点数 n_{12} 为整数, 所以 $n_{\text{补}}$ 是使上式整除的最小整数, 求得 $n_{\text{补}} = 2$ 。

此外, 题中为实现最佳归并, 应满足 12 叉哈夫曼树, $n = 120$, $m = 12$, 不满足 $(n - 1) \% (m - 1) = 0$ 的条件, 因此需要添加两个权值为 0 的叶结点, 使得 $n = 121$, 才能满足条件。